Algorithmisches Prinzip “Teile und Herrsche”

Manfred Hauswirth | Open Distributed Systems | Einführung in die Programmierung, WS 25/26

Rückblick

Technische Universität Berlin

VL 0 „Organisation und Inhalt“: Ablauf der Vorlesung, Termine
VL 1 „Algorithmen, Pseudocode, Sortieren I“: Insertion Sort
VL 2 „Algorithmen, Pseudocode, Sortieren II“: Selection Sort, Bubble Sort, Count Sort
VL 3 „Laufzeit und Speicherplatz“: Laufzeitanalyse der vorgestellten Sortierverfahren
VL 4 „Einfache Datenstrukturen“: Arrays, verkettete Listen, Structs in C, Stack, Queue
VL 5 „Bäume“: Binärbäume, Baumtraversierung, Laufzeitanalyse Baumoperationen
VL 6 „Teile und Herrsche I“: Einführung der algorithmischen Methode, Merge Sort
VL 7 „Korrektheitsbeweise“: Rechnermodel, Beispielbeweise
VL 8 „Dateien in C“: Dateien, Dateisysteme, Verzeichnisse, Dateiverwaltung mit C
VL 9 „Prioritätenschlangen/Halden/Heaps “: Heap Sort, Binärer Heap, Heap Operationen
VL 10 „Fortgeschrittene Sortierverfahren“: Quick Sort, Radix Sort
VL 11 „AVL Bäume“: Definition, Baumoperationen, Traversierung
VL 12 „Teile und Herrsche II“: Generalisierung des algorithmischen Prinzips, Mastertheorem
VL 13 „Q & A“: Offene Vorlesung/Wiederholung

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Prinzip *Teile & Herrsche (Divide & Conquer)*

- Teile Eingabe in mehrere Teile auf
- Löse das Problem rekursiv auf den Teilen
- Füge die Teillösungen zu einer Gesamtlösung zusammen

Open Distributed Sytems ODS

Teile & Herrsche – Mergesort

Technische Universität Berlin

Prinzip *Teile & Herrsche (Divide & Conquer)*

- Teile Eingabe in mehrere Teile auf
- Löse das Problem rekursiv auf den Teilen
- Füge die Teillösungen zu einer Gesamtlösung zusammen

Beispiel (Sortieren) – Mergesort

15	**7**	**6**	**13**	**25**	**4**	**9**	**12**

Open Distributed Sytems ODS

Teile & Herrsche – Mergesort

Prinzip *Teile & Herrsche (Divide & Conquer)*

- Teile Eingabe in mehrere Teile auf
- Löse das Problem rekursiv auf den Teilen
- Füge die Teillösungen zu einer Gesamtlösung zusammen

Beispiel (Sortieren) – Mergesort

Schritt 1:
Aufteilen der
Eingabe

Teile & Herrsche – Mergesort

Prinzip *Teile & Herrsche (Divide & Conquer)*

- Teile Eingabe in mehrere Teile auf
- Löse das Problem rekursiv auf den Teilen
- Füge die Teillösungen zu einer Gesamtlösung zusammen

Beispiel (Sortieren) – Mergesort

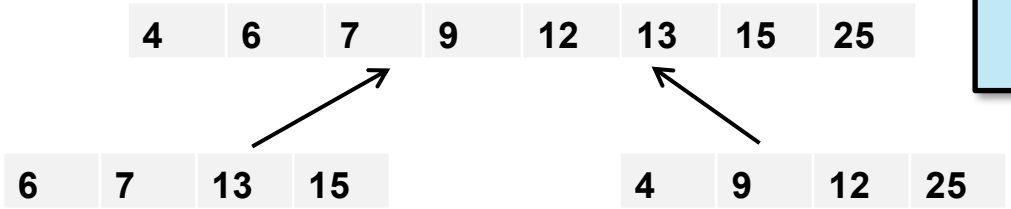
Schritt 2:
Rekursiv Sortieren

Teile & Herrsche – Mergesort

Prinzip *Teile & Herrsche (Divide & Conquer)*

- Teile Eingabe in mehrere Teile auf
- Löse das Problem rekursiv auf den Teilen
- Füge die Teillösungen zu einer Gesamtlösung zusammen

Beispiel (Sortieren) – Mergesort



Schritt 3:
Zusammenfügen

Teile & Herrsche

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

- Kann durch Laufzeitanalyse vorhergesagt werden

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

- Kann durch Laufzeitanalyse vorhergesagt werden
- Wovon hängt die Laufzeit ab?

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

- Kann durch Laufzeitanalyse vorhergesagt werden
- Wovon hängt die Laufzeit ab?
  - Anzahl der Teilprobleme

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

- Kann durch Laufzeitanalyse vorhergesagt werden
- Wovon hängt die Laufzeit ab?
  - Anzahl der Teilprobleme
  - Größe der Teilprobleme

Open Distributed Sytems
ODS

Teile & Herrsche

Technische Universität Berlin

Wodurch unterscheiden sich Teile & Herrsche Algorithmen?

- Die Anzahl der Teilprobleme
- Die Größe der Teilprobleme
- Den Algorithmus für das Zusammensetzen der Teilprobleme
- Den Rekursionsabbruch

Wann lohnt sich Teile & Herrsche?

- Kann durch Laufzeitanalyse vorhergesagt werden
- Wovon hängt die Laufzeit ab?
  - Anzahl der Teilprobleme
  - Größe der Teilprobleme
  - Algorithmus für das Zusammensetzen der Teilprobleme

Open Distributed Sytems
ODS

Rekursionsgleichung

Laufzeiten von Teile & Herrsche Algorithmen als Rekursionsgleichung der Form

$$T(n) = a \cdot T(n/b) + f(n)$$

(unter der Annahme: $T(1) = const$)

Rekursionsgleichung

Laufzeiten von Teile & Herrsche Algorithmen als Rekursionsgleichung der Form

$$T(n) = a \cdot T(n/b) + f(n)$$

Anzahl Unterprobleme

(unter der Annahme: T(1) = const)

Rekursionsgleichung

Laufzeiten von Teile & Herrsche Algorithmen als Rekursionsgleichung der Form

$$T(n) = a \cdot T(n/b) + f(n)$$

Anzahl Unterprobleme

Eingabegröße pro Unterproblem
(bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: $T(1) = const$)

Rekursionsgleichung

Laufzeiten von Teile & Herrsche Algorithmen als Rekursionsgleichung der Form

$$T(n) = a \cdot T(n/b) + f(n)$$

Anzahl Unterprobleme

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem
(bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: $T(1) = const$)

Rekursionsgleichung

Technische Universität Berlin

Laufzeiten von Teile & Herrsche Algorithmen als Rekursionsgleichung der Form

$$T(n) = a \cdot T(n/b) + f(n)$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem
(bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: $T(1) = \text{const}$)

Open Distributed Sytems
ODS

Rekursionsgleichung Mergesort

Beispiel: Laufzeiten von Mergesort

$$T(n) = 2 \cdot T(n/2) + c \cdot n$$

(unter der Annahme: n Zweierpotenz)

Rekursionsgleichung Mergesort

Beispiel: Laufzeiten von Mergesort

$$T(n) = 2 \cdot T(n/2) + c \cdot n$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem
(bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: n Zweierpotenz)

Teile & Herrsche – Binäre Suche

Problem: Finde Element b in sortiertem Feld

Teile & Herrsche – Binäre Suche

Problem: Finde Element b in sortiertem Feld

Algorithmus: Binäre Suche

Teile & Herrsche – Binäre Suche

Problem: Finde Element b in sortiertem Feld

Algorithmus: Binäre Suche

Prinzip:

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem: Finde Element b in sortiertem Feld

Algorithmus: Binäre Suche

Prinzip:

- Suche:
  - Vergleiche Element b mit dem mittleren Element des Feldes
  - Wenn gefunden, gib Feldindex zurück
  - Wenn kleiner, dann suche rekursiv im linken Teilfeld
  - Wenn größer, dann suche rekursiv im rechten Teilfeld

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem: Finde Element b in sortiertem Feld

Algorithmus: Binäre Suche

Prinzip:

- Suche:
  - Vergleiche Element b mit dem mittleren Element des Feldes
  - Wenn gefunden, gib Feldindex zurück
  - Wenn kleiner, dann suche rekursiv im linken Teilfeld
  - Wenn größer, dann suche rekursiv im rechten Teilfeld
- Abbruchbedingung:
  - Wenn Feldgröße == 0, gib „Element nicht gefunden“ zurück
  - Wenn gefunden, gib Feldindex zurück

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Problem: Finde Index des Elementes b in sortiertem Feld

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem:	Finde Index des Elementes b in sortiertem Feld
Annahme:	Element vorhanden, d.h. b existiert im sortierten Feld

Open Distributed Sytems
ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem:	Finde Index des Elementes b in sortiertem Feld
Annahme:	Element vorhanden, d.h. b existiert im sortierten Feld
Algorithmus:	Vereinfachte Binäre Suche

Open Distributed Sytems
ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem:	Finde Index des Elementes b in sortiertem Feld
Annahme:	Element vorhanden, d.h. b existiert im sortierten Feld
Algorithmus:	Vereinfachte Binäre Suche

Prinzip:

Open Distributed Sytems
ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem: Finde Index des Elementes b in sortiertem Feld

Annahme: Element vorhanden, d.h. b existiert im sortierten Feld

Algorithmus: Vereinfachte Binäre Suche

Prinzip:

- Suche:
  - Vergleiche Element b mit dem mittleren Element des Feldes
  - Wenn kleiner, dann suche rekursiv im linken Teilfeld
  - Wenn größer, dann suche rekursiv im rechten Teilfeld

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Problem: Finde Index des Elementes b in sortiertem Feld

Annahme: Element vorhanden, d.h. b existiert im sortierten Feld

Algorithmus: Vereinfachte Binäre Suche

Prinzip:

- Suche:
  - Vergleiche Element b mit dem mittleren Element des Feldes
  - Wenn kleiner, dann suche rekursiv im linken Teilfeld
  - Wenn größer, dann suche rekursiv im rechten Teilfeld
- Abbruchbedingung:
  - Wenn Feldgröße == 1, gib den Index zurück

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Aufruf: BinäreSuche(A,b,1,n)

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
p=1						r=7

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
p=1			q=4			r=7

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
				p=5		r=7

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
				p=5	q=6	r=7

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
				p=5	r=6	

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**
				p=5	r=6	
				q=5		

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**

p=5

r=5

Teile & Herrsche – Binäre Suche

Vereinfachte Binäre Suche mit Annahme b ist im Feld A

BinäreSuche(A,b,p,r) // A Feld, b Element, p Feldanfang, r Feldende

1. **if** p=r **then return** p
2. **else**
3. q ← ⌊(p+r)/2⌋
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Suche b=23

2	**7**	**10**	**11**	**23**	**34**	**47**

p=5

r=5

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion
- Annahme laut Satz: b ist in A[p..r] vorhanden

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion
- Annahme laut Satz: b ist in A[p..r] vorhanden
- Induktionsvoraussetzung:
  - b wird in einem Teilfeld gefunden Rekursionsschritt

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion
- Annahme laut Satz: b ist in A[p..r] vorhanden
- Induktionsvoraussetzung:
  - b wird in einem Teilfeld gefunden Rekursionsschritt
- Induktionsschritt:

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion
- Annahme laut Satz: b ist in A[p..r] vorhanden
- Induktionsvoraussetzung:
  - b wird in einem Teilfeld gefunden — Rekursionsschritt
- Induktionsschritt:
  - Finden des passenden Teilfeldes, durch Vergleich des mittleren Elementes mit b — Auswahlschritt

Binäre Suche: Beweisidee

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweisidee:

- Wir zeigen die Korrektheit per Induktion
- Annahme laut Satz: b ist in A[p..r] vorhanden
- Induktionsvoraussetzung:
  - b wird in einem Teilfeld gefunden — Rekursionsschritt
- Induktionsschritt:
  - Finden des passenden Teilfeldes, durch Vergleich des mittleren Elementes mit b — Auswahlschritt
  - Nutzen der Induktionsvoraussetzung

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.A.) Für n=0, d.h. p=r, gibt der Algorithmus p zurück. Dies ist der korrekte (weil einzige) Index.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.A.) Für n=0, d.h. p=r, gibt der Algorithmus p zurück. Dies ist der korrekte (weil einzige) Index.
- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.A.) Für n=0, d.h. p=r, gibt der Algorithmus p zurück. Dies ist der korrekte (weil einzige) Index.
- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt q≥p und q<r.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und $0 \leq m \leq n$ findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt $q \geq p$ und q<r.
  - Ist $b \leq A[q]$, so wird BinäreSuche rekursiv für A[p..q] aufgerufen.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und $0 \leq m \leq n$ findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt $q \geq p$ und q<r.
  - Ist $b \leq A[q]$, so wird BinäreSuche rekursiv für A[p..q] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[p..q].

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt q≥p und q<r.
  - Ist b≤A[q], so wird BinäreSuche rekursiv für A[p..q] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[p..q].
    - Damit folgt aus (I.V.), dass der Index von b gefunden wird.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt q≥p und q<r.
  - Ist b≤A[q], so wird BinäreSuche rekursiv für A[p..q] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[p..q].
    - Damit folgt aus (I.V.), dass der Index von b gefunden wird.
  - Ist b>A[q], so wird BinäreSuche rekursiv für A[q+1..r] aufgerufen.

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt q≥p und q<r.
  - Ist b≤A[q], so wird BinäreSuche rekursiv für A[p..q] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[p..q].
    - Damit folgt aus (I.V.), dass der Index von b gefunden wird.
  - Ist b>A[q], so wird BinäreSuche rekursiv für A[q+1..r] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[q+1..r].

Binäre Suche: Beweis

Satz: Algorithmus BinäreSuche(A,b,p,r) findet den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b in A[p..r] vorhanden ist.

Beweis: per Induktion über n=r-p

- (I.V.) Für alle r,p mit m=r-p und 0≤m≤n findet BinäreSuche(A,b,p,r) den Index einer Zahl b in einem sortierten Feld A[p..r], sofern b im Feld vorhanden ist.
- (I.S.) Wir betrachten den Aufruf von BinäreSuche für beliebige p, r mit n+1 = r-p.
  - Da n+1>0 folgt p<r und der Algorithmus führt den ersten **else**-Fall aus.
  - Dort wird q auf $\lfloor (p+r)/2 \rfloor$ gesetzt. – Es gilt q≥p und q<r.
  - Ist b≤A[q], so wird BinäreSuche rekursiv für A[p..q] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[p..q].
    - Damit folgt aus (I.V.), dass der Index von b gefunden wird.
  - Ist b>A[q], so wird BinäreSuche rekursiv für A[q+1..r] aufgerufen.
    - Da A[p..r] sortiert ist, liegt b in A[q+1..r].
    - Damit folgt aus (I.V.), dass der Index von b gefunden wird.

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Laufzeit:

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. q ← ⌊(p+r)/2⌋
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Laufzeit:

1

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)	Laufzeit:
1. **if** p=r **then return** p	1
2. **else**	1
3. q ← ⌊(p+r)/2⌋	
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)	
5. **else return** BinäreSuche(A,b,q+1,r)	

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)	Laufzeit:
1. **if** p=r **then return** p	1
2. **else**	1
3. q ← ⌊(p+r)/2⌋	1
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)	
5. **else return** BinäreSuche(A,b,q+1,r)	

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)	Laufzeit:
1. **if** p=r **then return** p	1
2. **else**	1
3. q ← ⌊(p+r)/2⌋	1
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)	1+T(⌊n/2⌋)
5. **else return** BinäreSuche(A,b,q+1,r)	

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)	Laufzeit:
1. **if** p=r **then return** p	1
2. **else**	1
3. q ← ⌊(p+r)/2⌋	1
4. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q)	1+T(⌊n/2⌋)
5. **else return** BinäreSuche(A,b,q+1,r)	1+T(⌈n/2⌉)

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Laufzeit:

1	
1	
1	
$1+T(\lfloor n/2 \rfloor)$	
$1+T(\lceil n/2 \rceil)$	
$5+\max\{T(\lfloor n/2 \rfloor), T(\lceil n/2 \rceil)\}$	

Binäre Suche: Laufzeit

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Laufzeit:

1	
1	
1	
$1+T(\lfloor n/2 \rfloor)$	
$1+T(\lceil n/2 \rceil)$	
$5+\max\{T(\lfloor n/2 \rfloor), T(\lceil n/2 \rceil)\}$	

Laufzeit: $T(n) = \{$

Binäre Suche: Laufzeit

Technische Universität Berlin

BinäreSuche(A,b,p,r)	Laufzeit:
1. **if** p=r **then return** p	1
2. **else**	1
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$	1
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)	$1+T(\lfloor n/2 \rfloor)$
5. **else return** BinäreSuche(A,b,q+1,r)	$1+T(\lceil n/2 \rceil)$
	$5+ \max\{T(\lfloor n/2 \rfloor), T(\lceil n/2 \rceil)\}$

Laufzeit: $T(n) = \begin{cases} 1 & \text{, falls } n=1 \\ \end{cases}$

Open Distributed Sytems ODS

Binäre Suche: Laufzeit

Technische Universität Berlin

BinäreSuche(A,b,p,r)

1. **if** p=r **then return** p
2. **else**
3. $q \leftarrow \lfloor (p+r)/2 \rfloor$
4. **if** $b \leq A[q]$ **then return** BinäreSuche(A,b,p,q)
5. **else return** BinäreSuche(A,b,q+1,r)

Laufzeit:

1	
1	
1	
$1+T(\lfloor n/2 \rfloor)$	
$1+T(\lceil n/2 \rceil)$	
$5+ \max\{T(\lfloor n/2 \rfloor), T(\lceil n/2 \rceil)\}$	

Laufzeit: $T(n) = \begin{cases} 1 & \text{, falls } n=1 \\ 5+ \max\{T(\lceil n/2 \rceil), T(\lfloor n/2 \rfloor)\} & \text{, falls } n>1 \end{cases}$

Open Distributed Sytems ODS

Binäre Suche: Rekursionsgleichung

Beispiel: Laufzeiten von Binäre Suche

$$T(n) = 1 \cdot T(n/2) + c$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem
(bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: n Zweierpotenz)

Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

Intuition: wir ignorieren Runden

Binäre Suche: Rekursionsgleichung

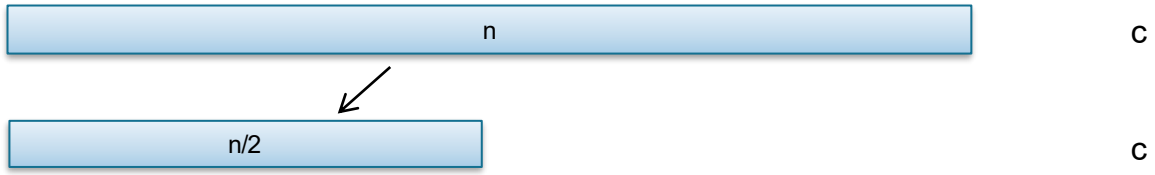
Auflösen von $T(n) \leq T(n/2) + c$

Intuition: wir ignorieren Runden

Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

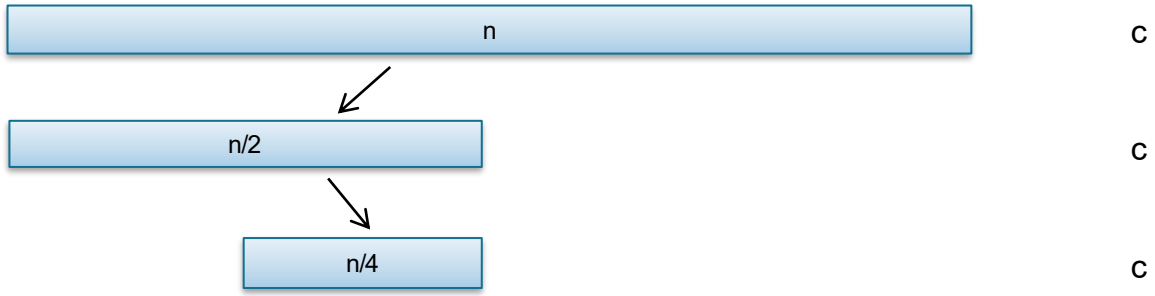
Intuition: wir ignorieren Runden



Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

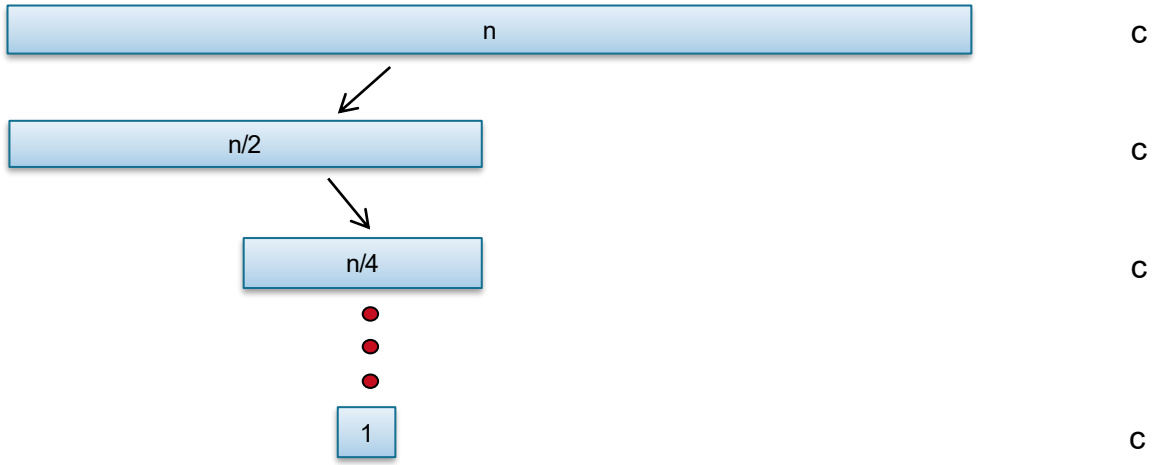
Intuition: wir ignorieren Runden



Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

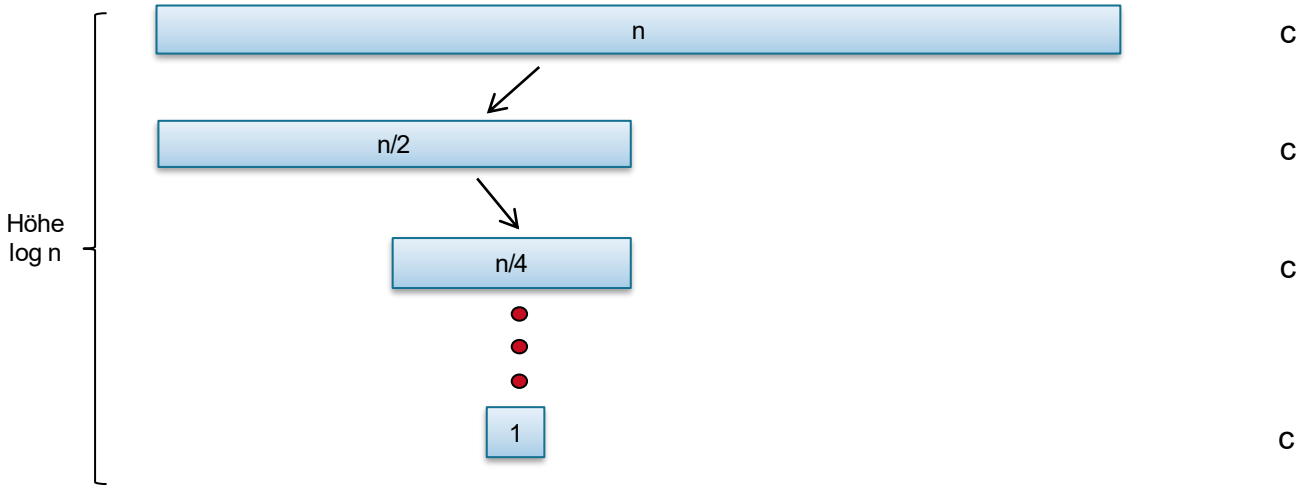
Intuition: wir ignorieren Runden



Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

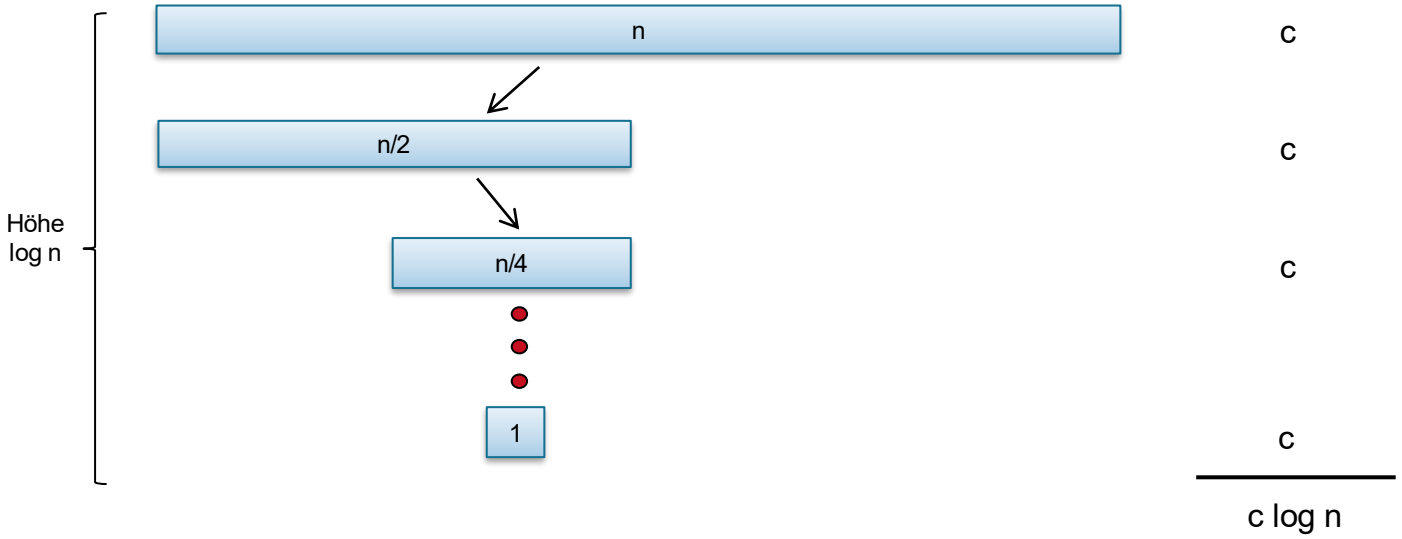
Intuition: wir ignorieren Runden



Binäre Suche: Rekursionsgleichung

Auflösen von $T(n) \leq T(n/2) + c$

Intuition: wir ignorieren Runden



Binäre Suche: Laufzeit

Technische Universität Berlin

Satz: Algorithmus BinäreSuche hat eine Laufzeit von $O(\log n)$.

Beweis: per Induktion

- Zu zeigen ist $T(n) \leq 5 \lceil \log n \rceil + 1$.

Open Distributed Sytems ODS

Binäre Suche:Laufzeit

Technische Universität Berlin

Binäre Suche vs. lineare Suche

Laufzeit	10	100	1,000	10,000	100,000
n	10	100	1,000	10,000	100,000
log n	3	6	10	13	17

Beobachtung

- n wächst sehr viel stärker als log n
- Binäre Suche effizient für riesige Datenmengen
- In der Praxis ist log n fast wie eine Konstante

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Binärsuche ohne Annahme

Problem: Finde Element b in sortiertem Feld

Eingabe: Sortiertes Feld A, gesuchtes Element b

Ausgabe: Index i mit A[i] = b oder -1 (für nicht gefunden)

BinäreSuche(A,b,p,r) // A Feld, b Element, p, r Feldanfang, -ende

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Binärsuche ohne Annahme

Problem: Finde Element b in sortiertem Feld

Eingabe: Sortiertes Feld A, gesuchtes Element b

Ausgabe: Index i mit A[i] = b oder -1 (für nicht gefunden)

BinäreSuche(A,b,p,r) // A Feld, b Element, p, r Feldanfang, -ende

1. **if** p=r **and** A[p] == b **then return** p // Element gefunden

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Binärsuche ohne Annahme

Problem: Finde Element b in sortiertem Feld

Eingabe: Sortiertes Feld A, gesuchtes Element b

Ausgabe: Index i mit A[i] = b oder -1 (für nicht gefunden)

BinäreSuche(A,b,p,r) // A Feld, b Element, p, r Feldanfang, -ende

1. **if** p=r **and** A[p] == b **then return** p // Element gefunden
2. **else if** p=r **and** A[p] != b **then return** -1 // Element nicht gefunden

Open Distributed Sytems ODS

Teile & Herrsche – Binäre Suche

Technische Universität Berlin

Binärsuche ohne Annahme

Problem: Finde Element b in sortiertem Feld

Eingabe: Sortiertes Feld A, gesuchtes Element b

Ausgabe: Index i mit A[i] = b oder -1 (für nicht gefunden)

BinäreSuche(A,b,p,r) // A Feld, b Element, p, r Feldanfang, -ende

1. **if** p=r **and** A[p] == b **then return** p // Element gefunden
2. **else if** p=r **and** A[p] != b **then return** -1 // Element nicht gefunden
3. **else**
4. q ← ⌊(p+r)/2⌋
5. **if** b ≤ A[q] **then return** BinäreSuche(A,b,p,q) // links
6. **else return** BinäreSuche(A,b,q+1,r) // rechts

Open Distributed Sytems ODS

Teile & Herrsche – Integer Multiplikation

Technische Universität Berlin

Problem: Multipliziere zwei n-Bit Integer

Eingabe: Zwei n-Bit Integer X,Y

Ausgabe: 2n-Bit Integer Z mit Z=XY

Annahmen:

- Wir können n-Bit Integer in $\Theta(n)$ (worst case) Zeit addieren
- Wir können n-Bit Integer in $\Theta(n+k)$ (worst case) Zeit mit 2^k multiplizieren (durch Shift)

Open Distributed Sytems ODS

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$1101 \cdot 1011$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$\underline{1101 \cdot 1011}$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$\begin{array}{r} 1101 \cdot 1011 \\ \hline 1101 \end{array}$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$
\begin{array}{r}
1101 \cdot 1011 \\
\hline
1101 \\
1101 \\
\end{array}
$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$
\begin{array}{r}
1101 \cdot 1011 \\
\hline
1101 \\
1101 \\
0000
\end{array}
$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

$$
\begin{array}{r}
1101 \cdot 1011 \\
\hline
1101 \\
1101 \\
0000 \\
1101
\end{array}
$$

Integer Multiplikation: Schulmethode

Schulmethode: (13·11)

```
 1101 · 1011
 ----------
       1101
      1101
     0000
    1101
 ----------
   10001111
```

Integer Multiplikation: Schulmethode

Laufzeit Schulmethode

- n Multiplikationen mit 2^k für ein $k \leq n$
- n-1 Additionen im worst-case:

$$\underbrace{11\ldots111}_{\text{n-Bit}} \cdot \underbrace{11\ldots111}_{\text{n-Bit}}$$

Integer Multiplikation: Schulmethode

Laufzeit Schulmethode

- n Multiplikationen mit 2^k für ein $k \leq n$
- n-1 Additionen im worst-case:

$$\underbrace{11\ldots111}_{\text{n-Bit}} \cdot \underbrace{11\ldots111}_{\text{n-Bit}}$$

- Jede Addition $\Theta(n)$ Zeit

Integer Multiplikation: Schulmethode

Laufzeit Schulmethode

- n Multiplikationen mit 2^k für ein $k \leq n$
- n-1 Additionen im worst-case:

$$\underbrace{11\ldots111}_{\text{n-Bit}} \cdot \underbrace{11\ldots111}_{\text{n-Bit}}$$

- Jede Addition $\Theta(n)$ Zeit
- Insgesamt $\Theta(n^2)$ Laufzeit

Integer Multiplikation: Schulmethode

Laufzeit Schulmethode

- n Multiplikationen mit 2^k für ein $k \leq n$
- n-1 Additionen im worst-case:

$$\underbrace{11\ldots111}_{\text{n-Bit}} \cdot \underbrace{11\ldots111}_{\text{n-Bit}}$$

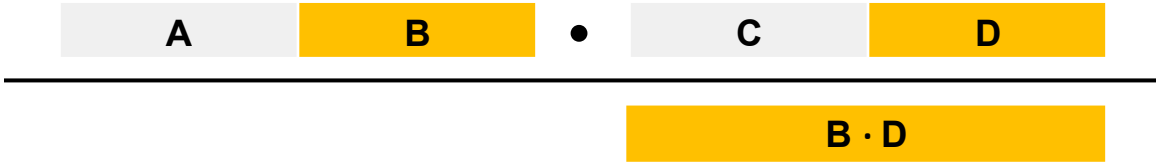
- Jede Addition $\Theta(n)$ Zeit
- Insgesamt $\Theta(n^2)$ Laufzeit

Bessere Laufzeit mit Teile & Herrsche?

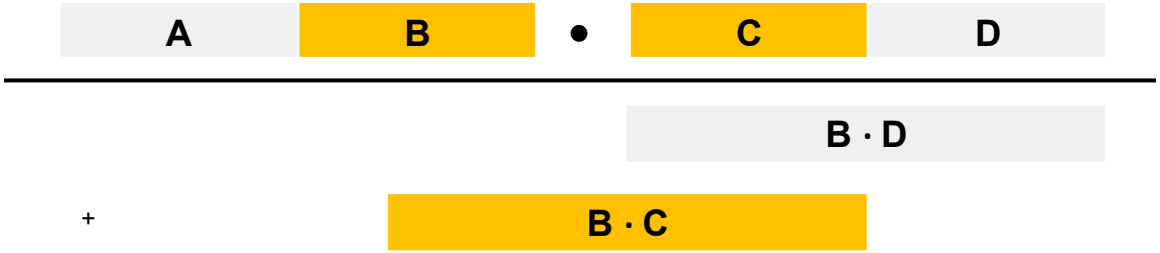
Integer Multiplikation mit Teile & Herrsche – Erster Versuch

A	**B**	•	**C**	**D**

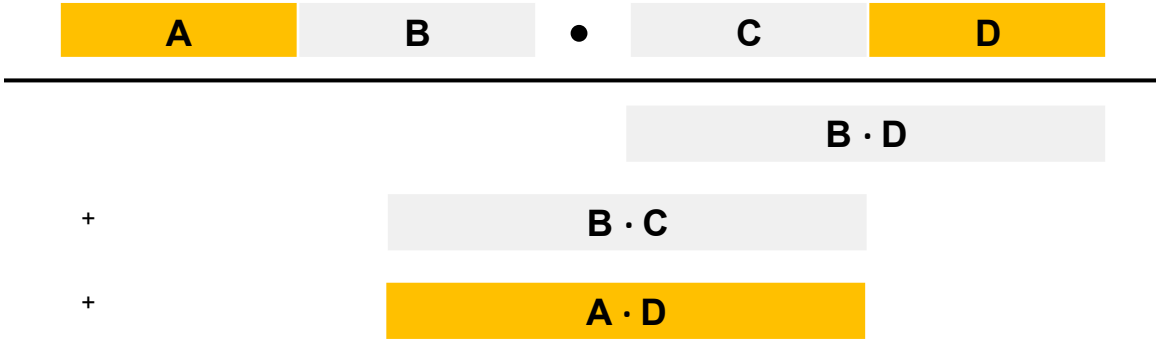
Integer Multiplikation mit Teile & Herrsche – Erster Versuch



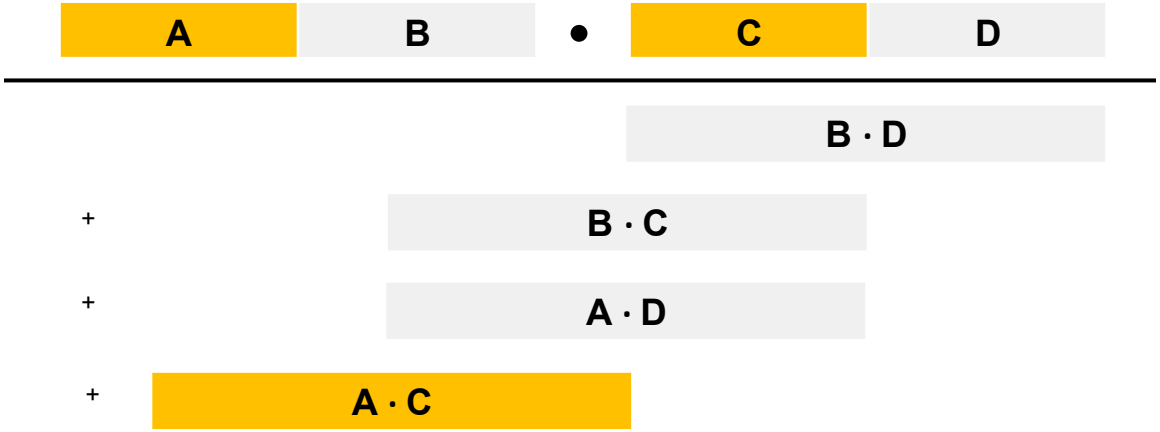
Integer Multiplikation mit Teile & Herrsche – Erster Versuch



Integer Multiplikation mit Teile & Herrsche – Erster Versuch



Integer Multiplikation mit Teile & Herrsche – Erster Versuch



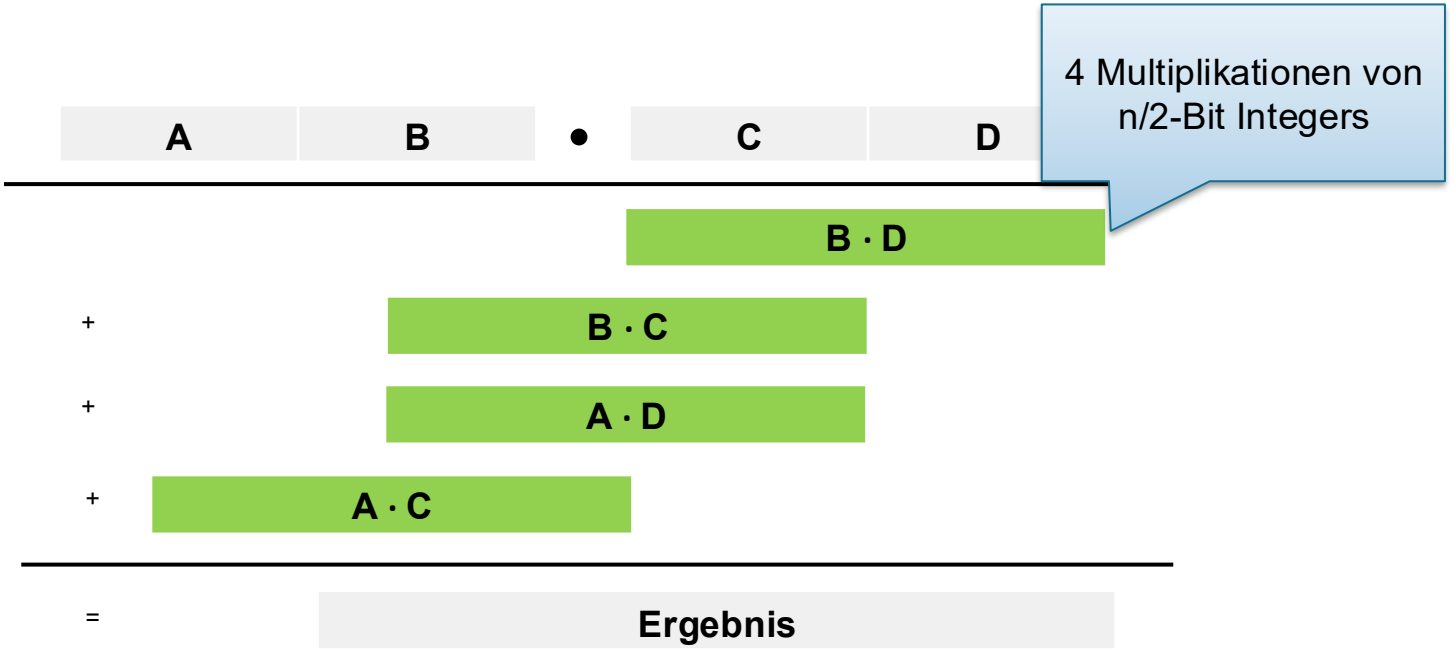
Integer Multiplikation mit Teile & Herrsche – Erster Versuch

Technische Universität Berlin

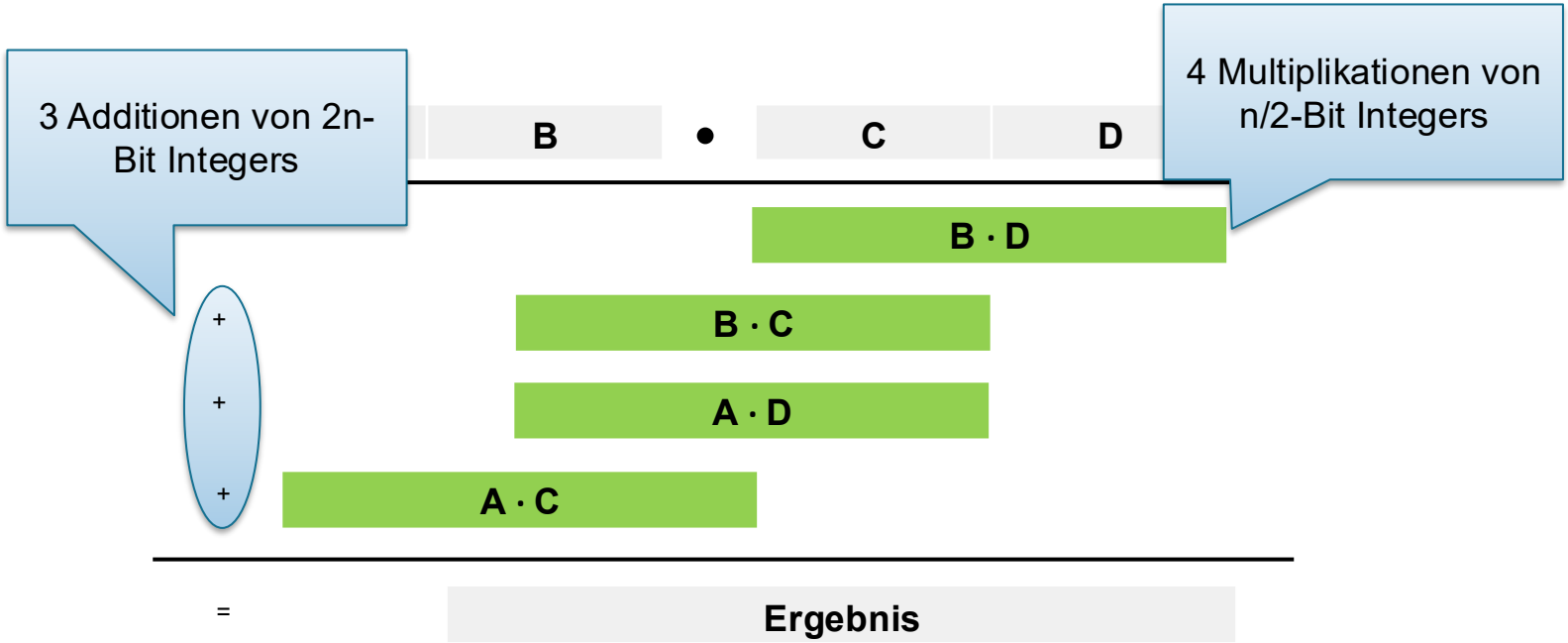
	A	B	•	C	D
				B · D
+		B · C		
+		A · D		
+	A · C			
=	Ergebnis			

Open Distributed Sytems ODS

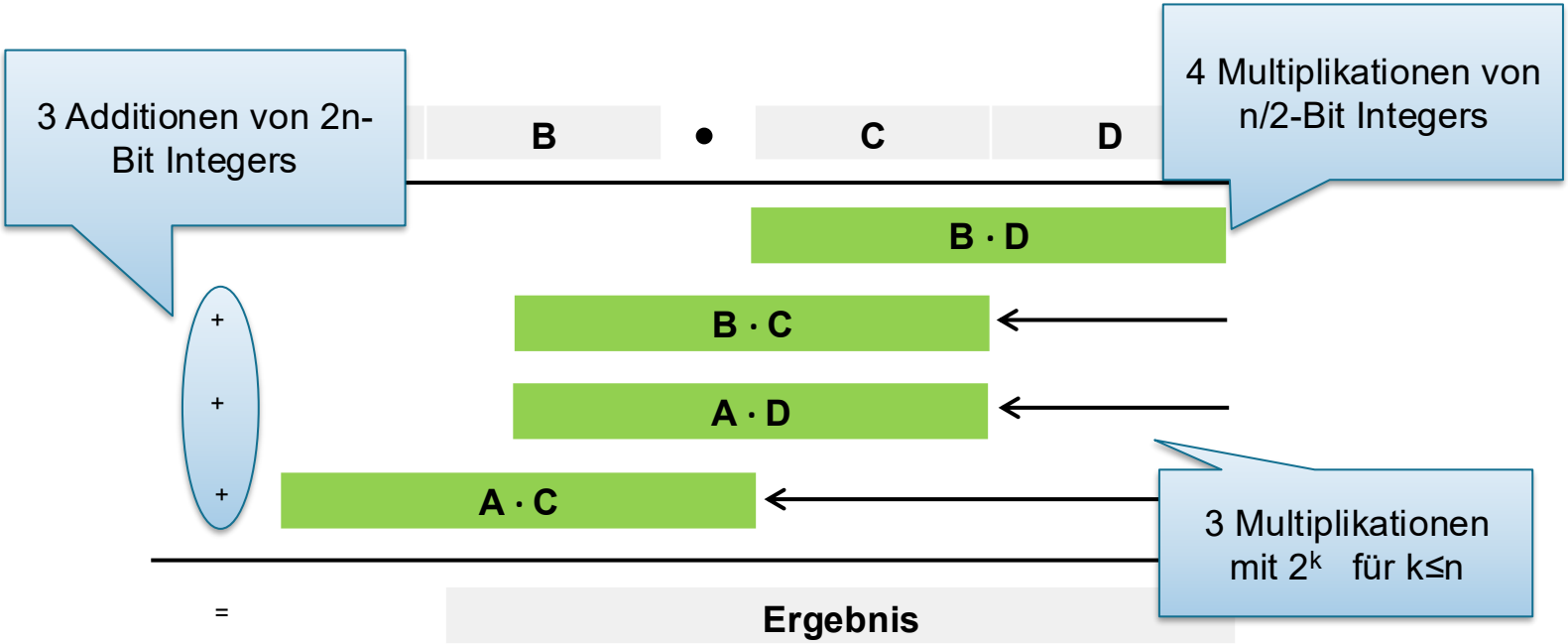
Integer Multiplikation mit Teile & Herrsche – Erster Versuch



Integer Multiplikation mit Teile & Herrsche – Erster Versuch



Integer Multiplikation mit Teile & Herrsche – Erster Versuch



Rekursionsgleichung: Integer Multiplikation

Teile und Herrsche – Erster Versuch

$$T(n) = 4 \cdot T(n/2) + cn$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem (bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: n Zweierpotenz, T(1) = const.)

Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Rekursionsgleichung: Integer Multiplikation

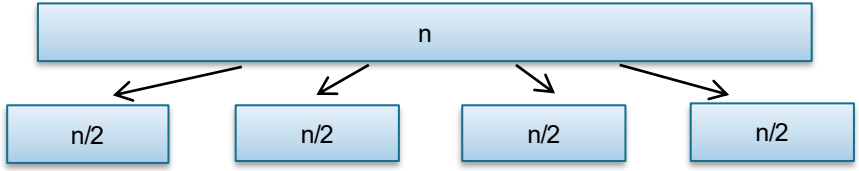
Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

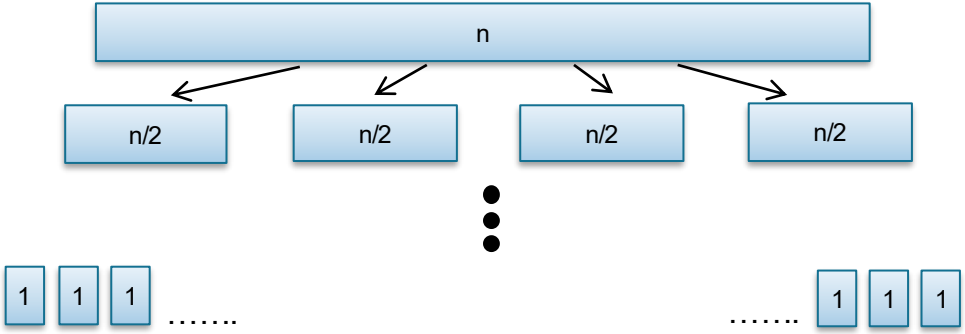
$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

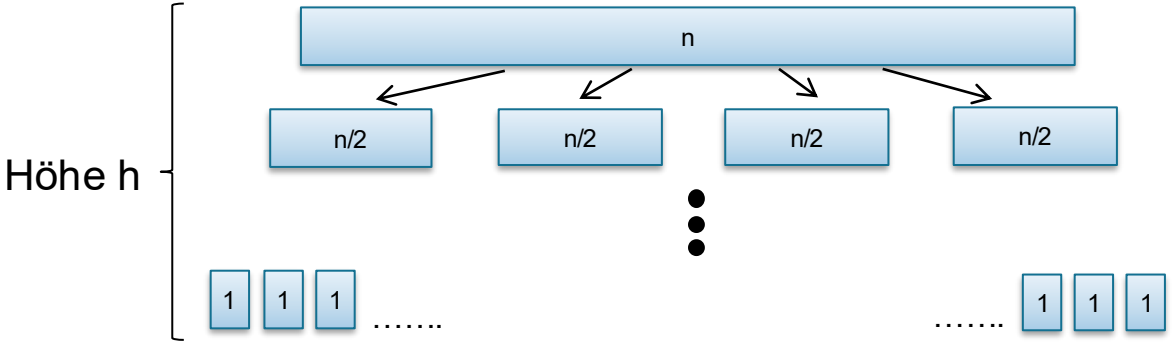
$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

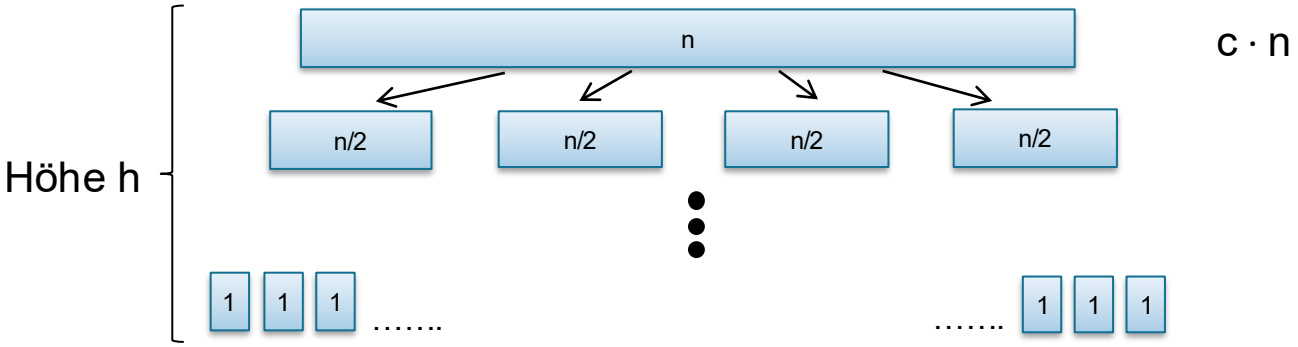
$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

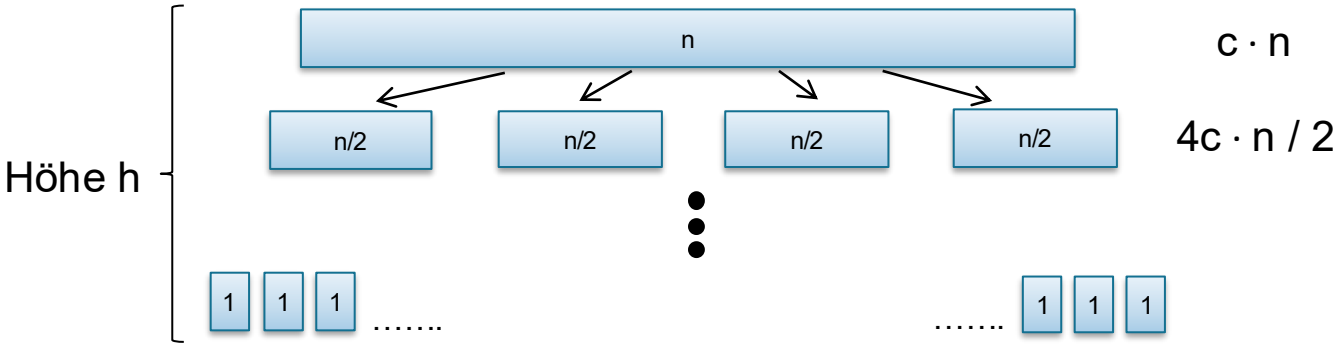
$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

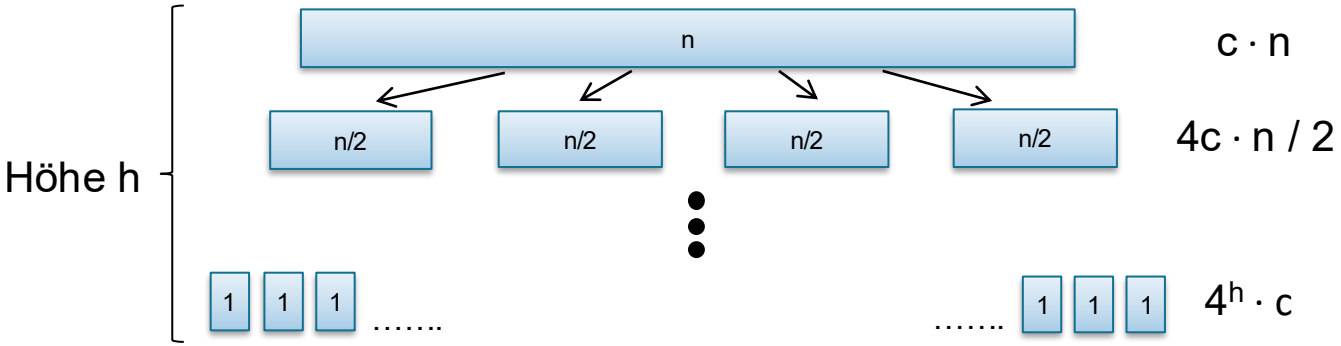
$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

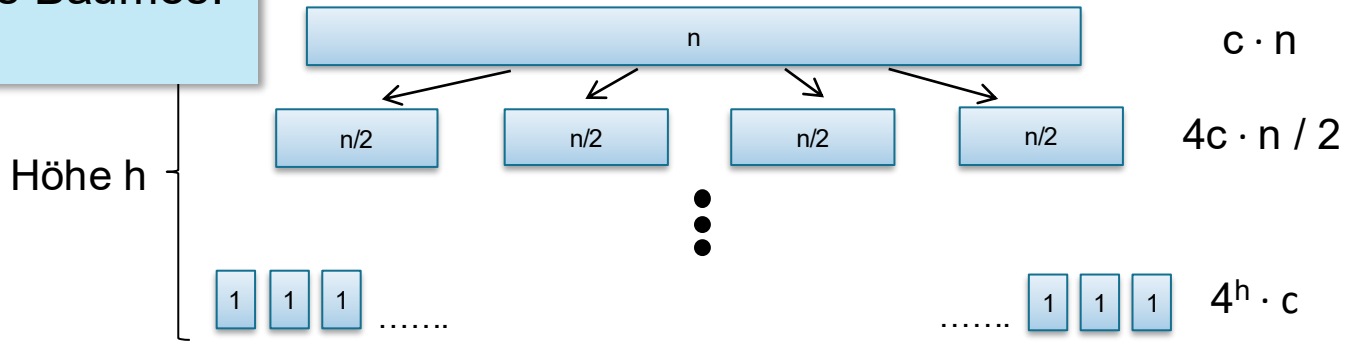


Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$

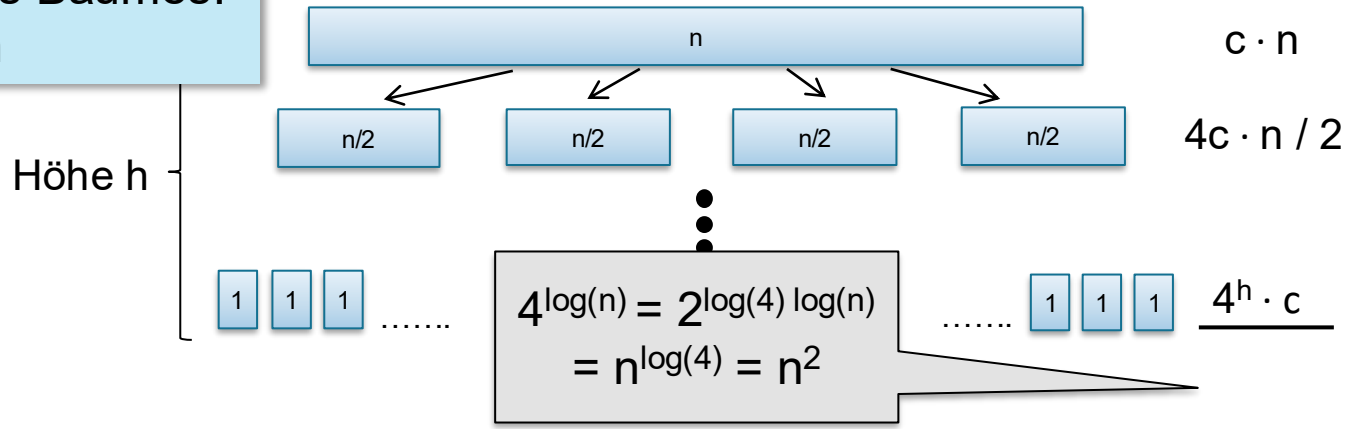


Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$

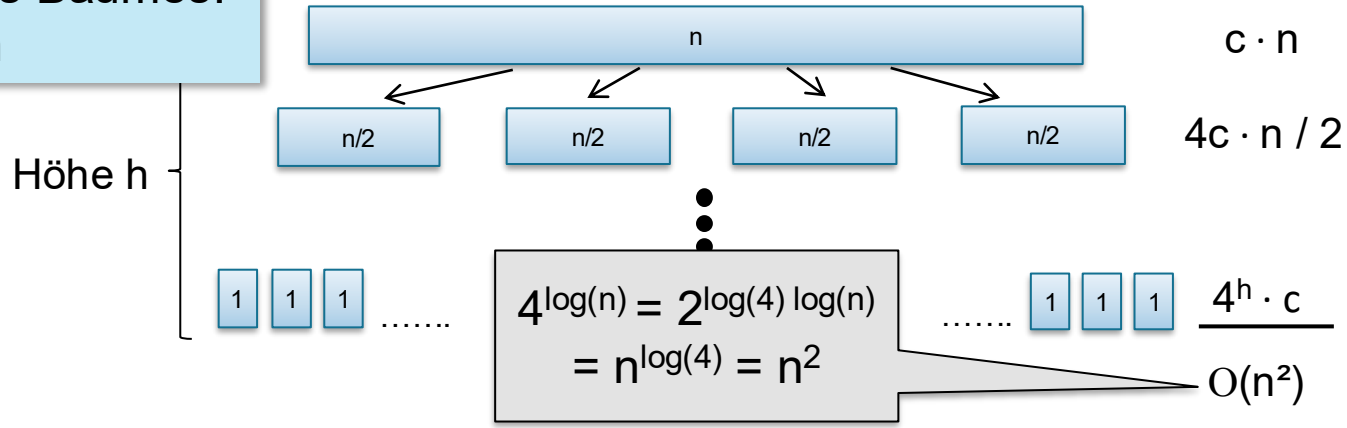


Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$

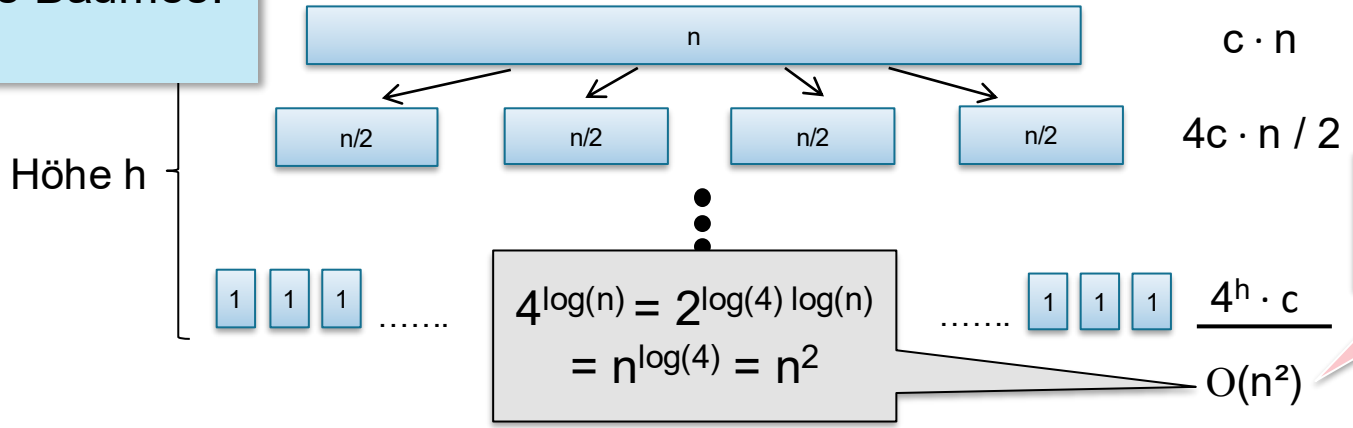


Rekursionsgleichung: Integer Multiplikation

Laufzeit einfaches Teile & Herrsche

$$T(n) \leq \begin{cases} 4\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$



$c \cdot n$

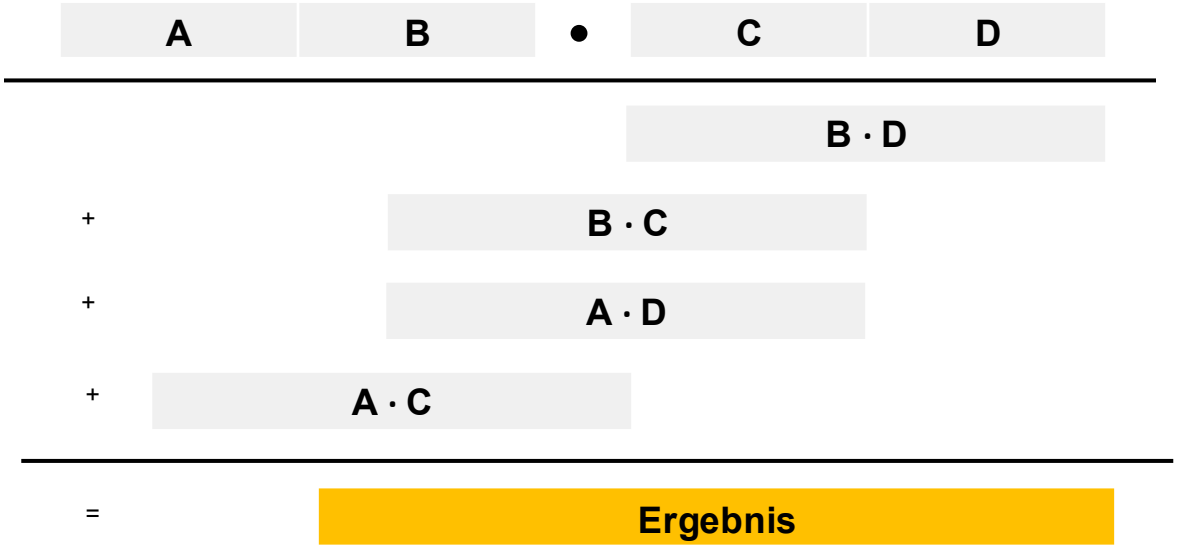
$4c \cdot n / 2$

$4^h \cdot c$

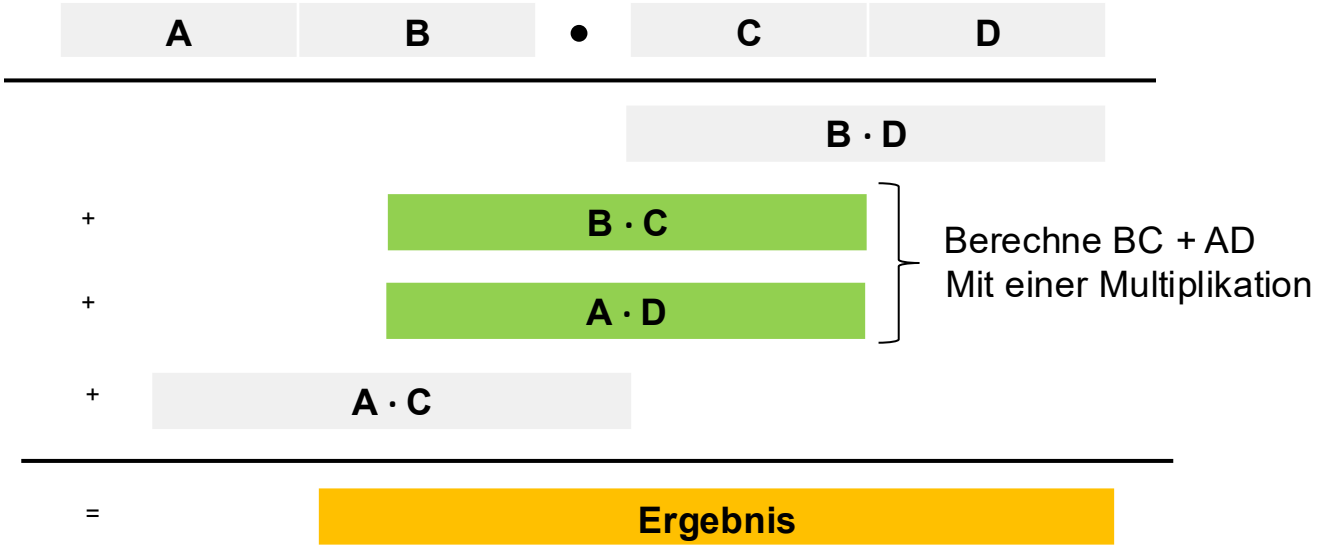
$O(n^2)$

Nichts gewonnen!

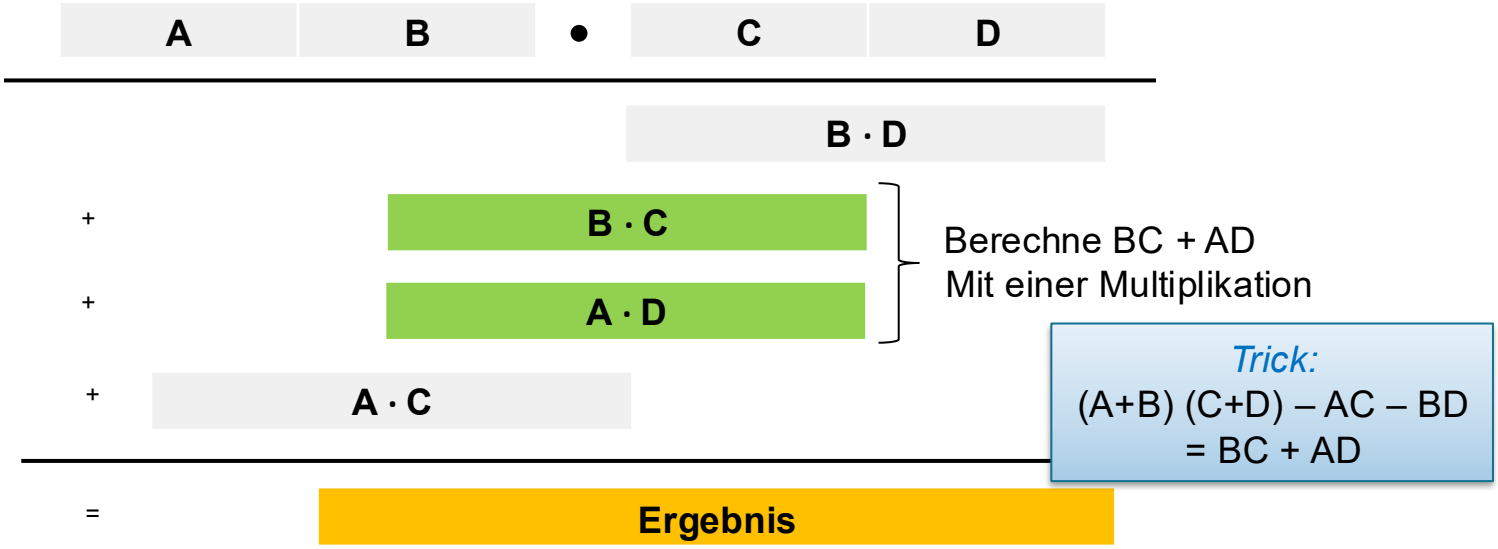
Integer Multiplikation mit Teile & Herrsche – Zweiter Versuch



Integer Multiplikation mit Teile & Herrsche – Zweiter Versuch



Integer Multiplikation mit Teile & Herrsche – Zweiter Versuch



Rekursionsgleichung: Integer Multiplikation

Teile und Herrsche – Zweiter Versuch

$$T(n) = 3 \cdot T(n/2) + cn$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

Eingabegröße pro Unterproblem (bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: n Zweierpotenz, T(1) = const.)

Rekursionsgleichung: Integer Multiplikation

Teile und Herrsche – Zweiter Versuch

- 3 Multiplikationen der Länge n/2
- [AC, BD, (A+B) (C+D)]
- Konstant viele Additionen und Multiplikationen mit Zweierpotenzen

Laufzeit: $T(n) = \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$

Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Rekursionsgleichung: Integer Multiplikation

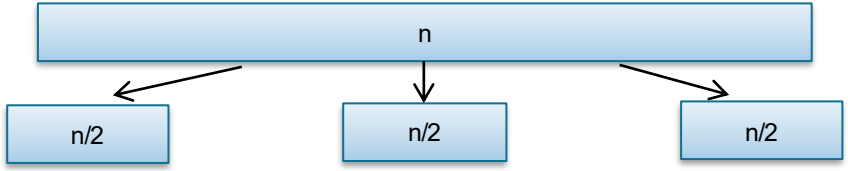
Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

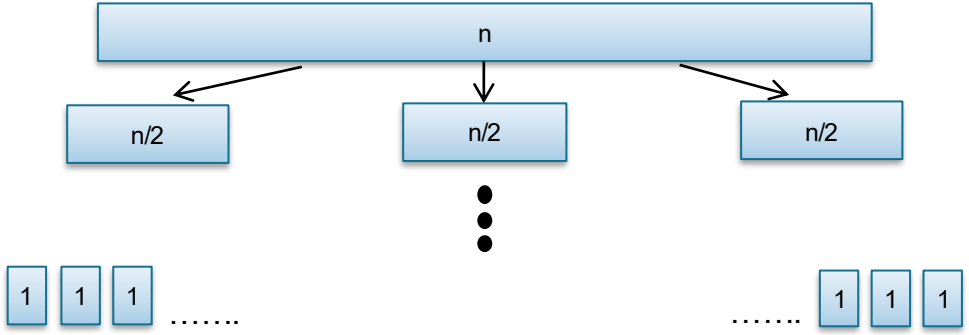
$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

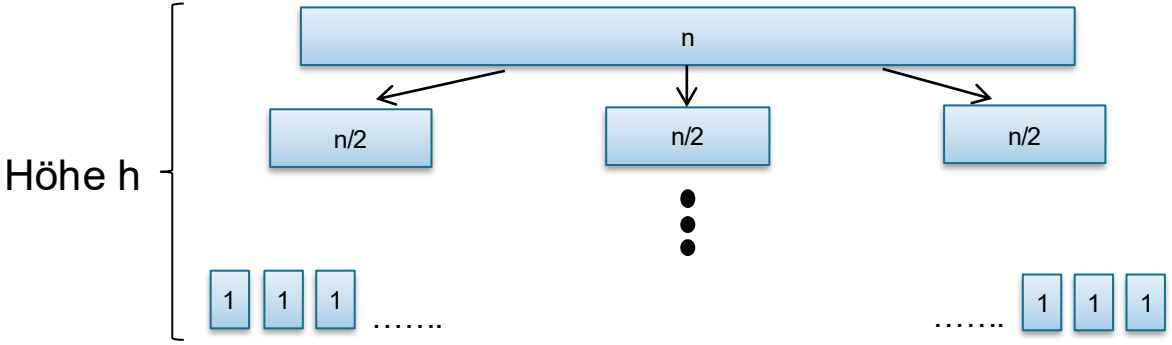
$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

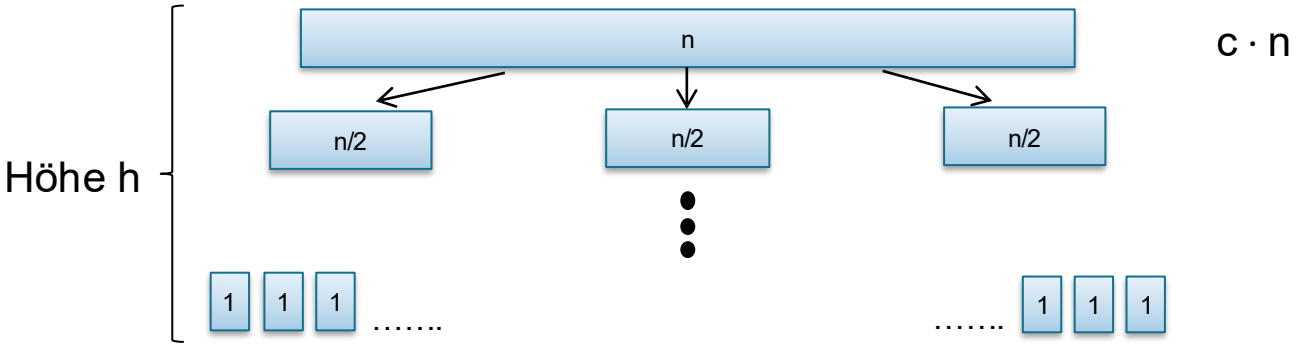
$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

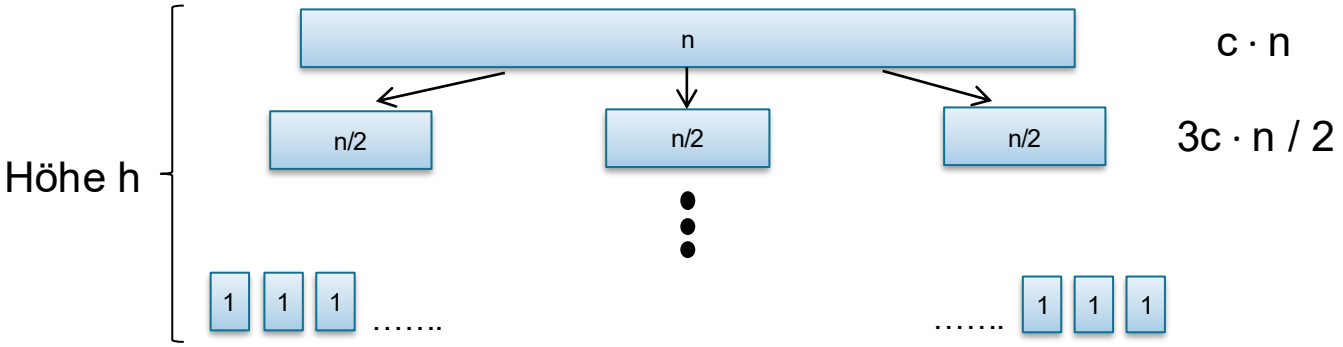
$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

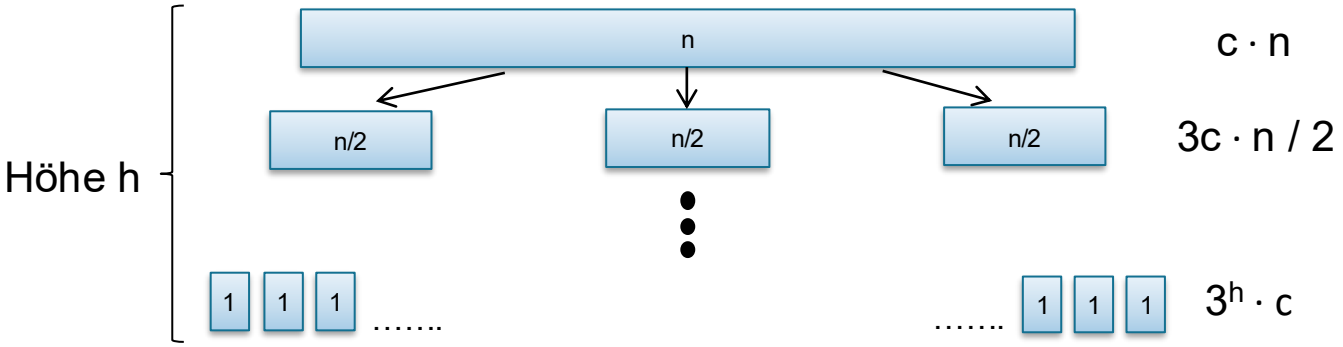
$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$



Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

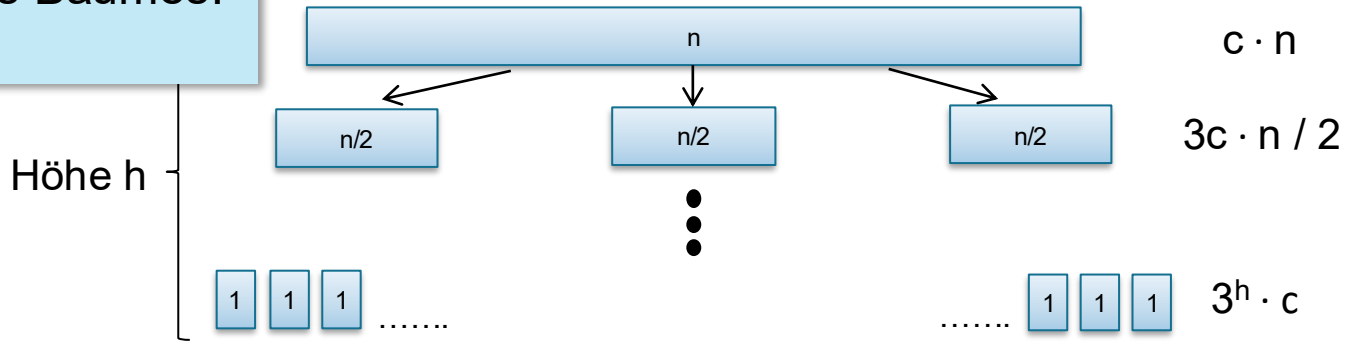


Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$

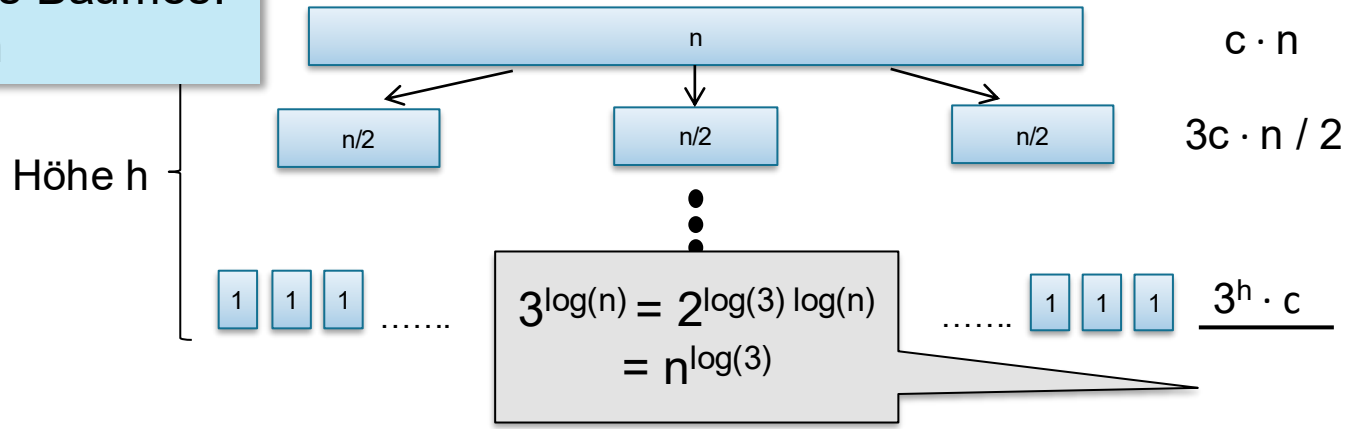


Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$



$c \cdot n$

$3c \cdot n / 2$

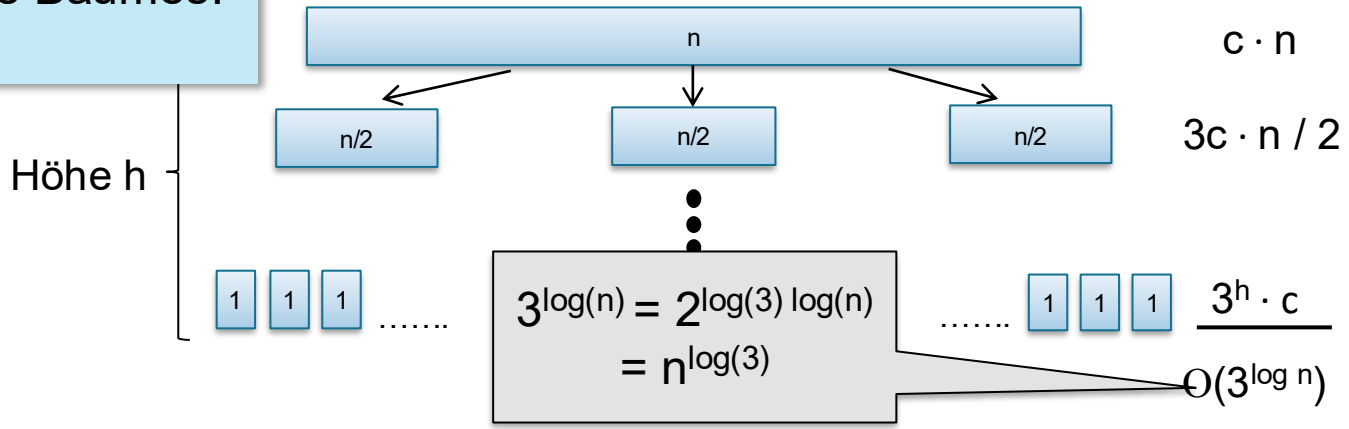
$3^h \cdot c$

Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$

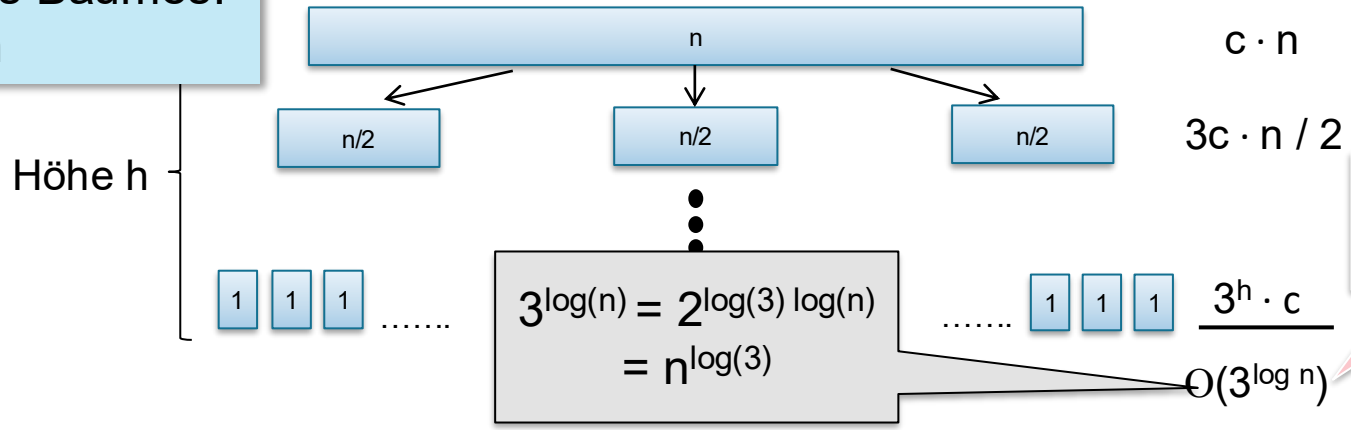


Rekursionsgleichung: Integer Multiplikation

Laufzeit verbessertes Teile & Herrsche

$$T(n) \leq \begin{cases} 3\,T(n/2) + cn & , n>1 \\ c & , n=1 \text{ für geeignete Konstante } c \end{cases}$$

Höhe des Baumes:
$h = \log n$



$O(n^{1,59})$ gewonnen!

Laufzeitgewinn

Laufzeit	10	100	1,000
n^2	100	10,000	1.000,000
$n^{1,59}$	39	1,514	58,885
Faktor	2.56	6.6	17

Integer Multiplikation mit Teile & Herrsche – Zweiter Versuch

Technische Universität Berlin

Satz: Die Laufzeit der verbesserten Integer Multiplikation ist

$$O(3^{\log n}) = O(n^{\log 3}) = O(n^{1,59}).$$

Beweis: Per Induktion über n.

- Zu zeigen ist, dass $T(n) \leq c\, 3^{\log n} - 2cn$.

Open Distributed Sytems ODS

Teile & Herrsche: Allgemeine Rekursionsgleichung

Teile und Herrsche – Zweiter Versuch

$$T(n) = a \cdot T(n/b) + f(n)$$

Anzahl Unterprobleme

Aufwand für Aufteilen und Zusammenfügen

Aufwand pro Unterproblem

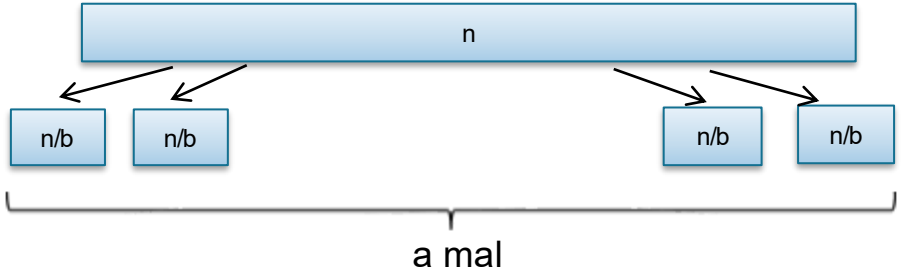
Eingabegröße pro Unterproblem (bestimmt Höhe des Rekursionsbaums)

(unter der Annahme: n Zweierpotenz, T(1) = const.)

Teile & Herrsche: Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ 1 & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.

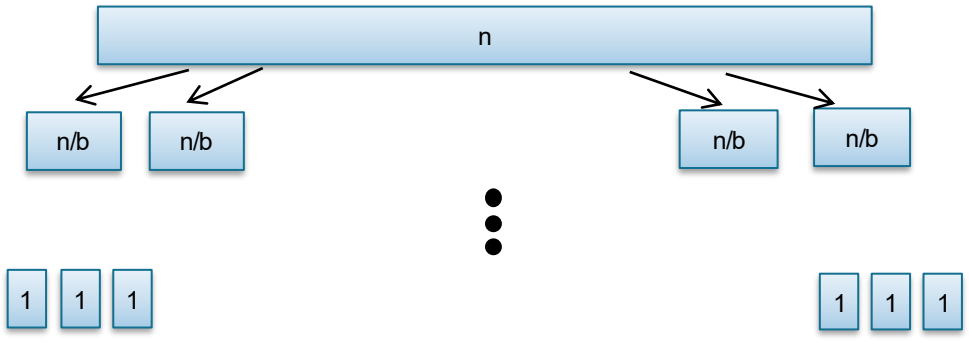


Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.

Höhe
$h=\log_b(n)$

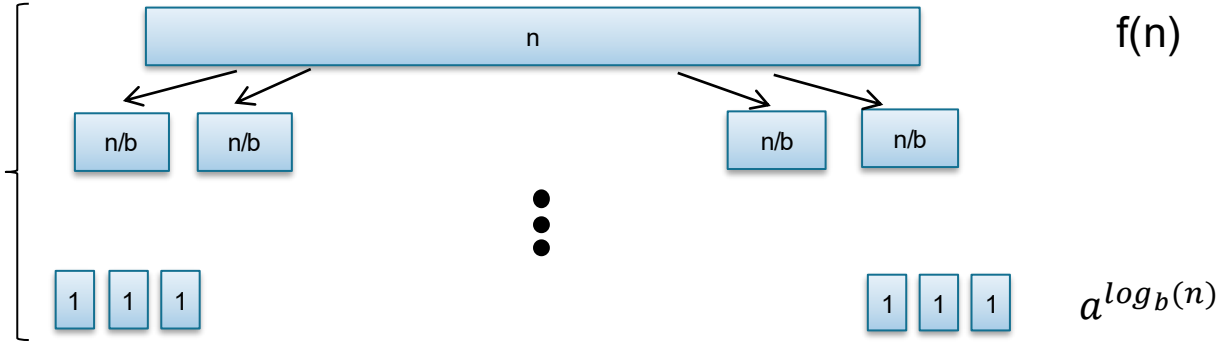


$a^{\log_b(n)}$

Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

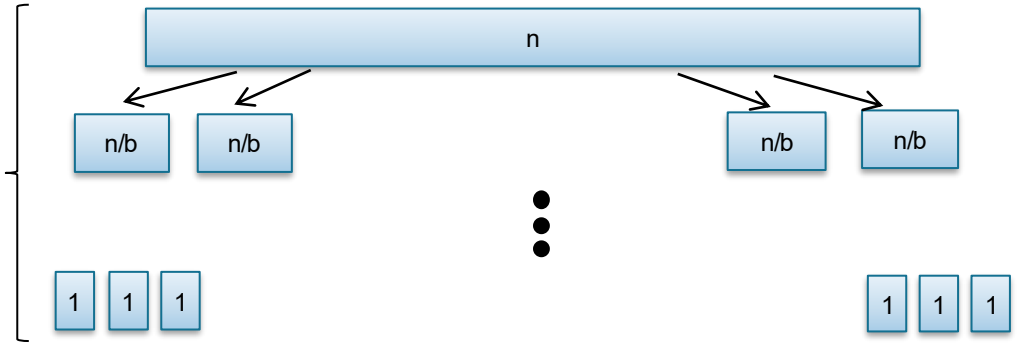
$f(n) = O(n^k)$ für Konstante k.



Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

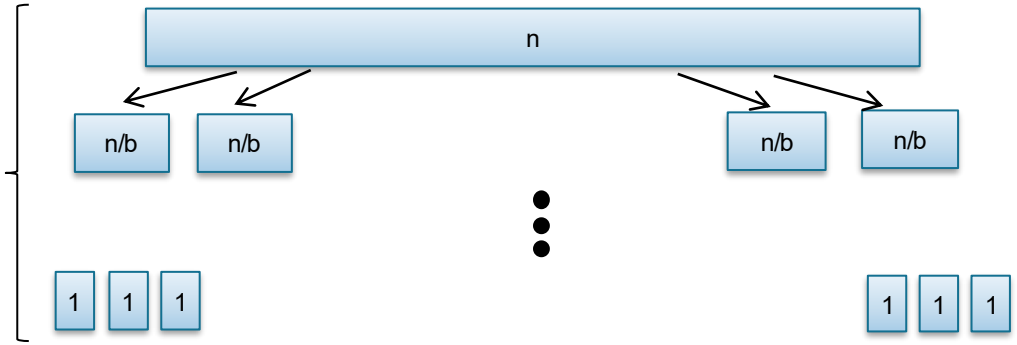
$f(n) = O(n^k)$ für Konstante k.



Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.



$f(n)$

$a \cdot f(n/b)$

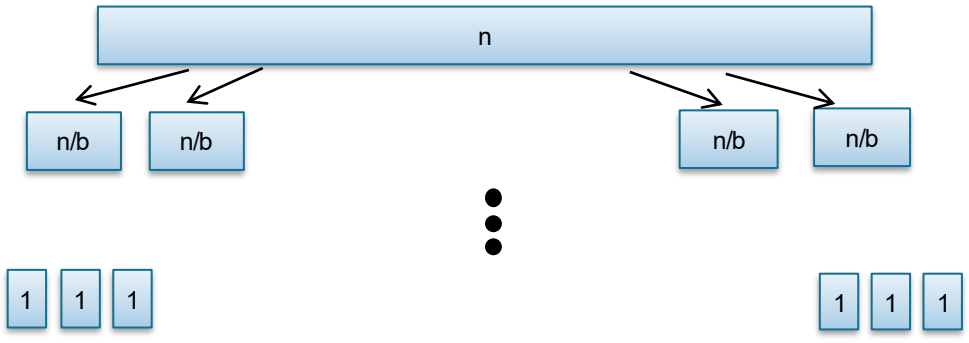
$a^{log_b(n)}$

Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.

Höhe
$h=\log_b(n)$



$f(n)$

$a \cdot f(n/b)$

$a^{\log_b(n)}$

?

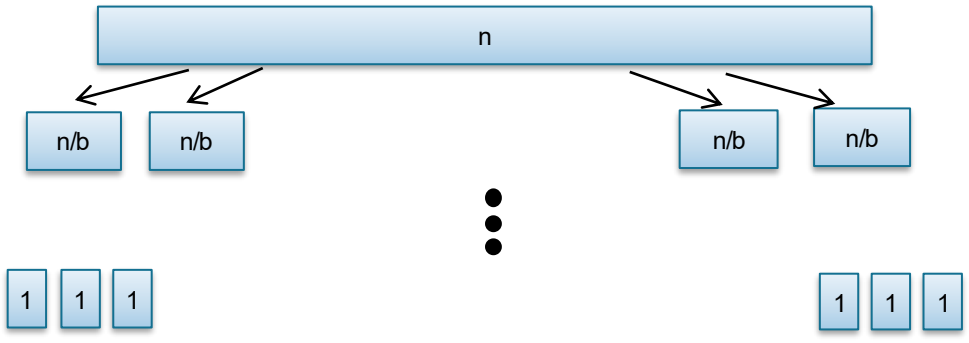
Teile & Herrsche
Allgemeine Rekursionsgleichung

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.

Für viele Fälle erlaubt das Mastertheorem eine Aussage über die Laufzeit.

Höhe
$h=\log_b(n)$



$f(n)$

$a \cdot f(n/b)$

$a^{\log_b(n)}$

?

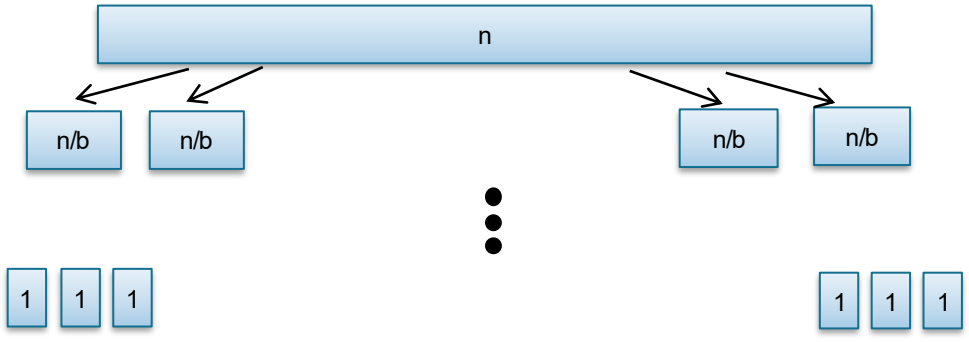
Teile & Herrsche
Mastertheorem

$$T(n) = \begin{cases} a\,T(n/b) + f(n) & , n>1 \\ \Theta(1) & , n=1 \end{cases}$$

$f(n) = O(n^k)$ für Konstante k.

Fall 1	Fall 2	Fall 3
Laufzeit auf unterster Rekursionsstufe dominiert	Laufzeiten Zusammensetzen und Rekursion vergleichbar	Laufzeit Zusammensetzen dominiert
$O(n^{log_b(a)})$	$O(n^{log_b(a)} \cdot \log(n))$	$O(f(n))$

Höhe
$h=\log_b(n)$



f(n)

a· f(n/b)

$a^{log_b(n)}$

?

Ausblick

VL 0 „Organisation und Inhalt“: Ablauf der Vorlesung, Termine
VL 1 „Algorithmen, Pseudocode, Sortieren I“: Insertion Sort
VL 2 „Algorithmen, Pseudocode, Sortieren II“: Selection Sort, Bubble Sort, Count Sort
VL 3 „Laufzeit und Speicherplatz“: Laufzeitanalyse der vorgestellten Sortierverfahren
VL 4 „Einfache Datenstrukturen“: Arrays, verkettete Listen, Structs in C, Stack, Queue
VL 5 „Bäume“: Binärbäume, Baumtraversierung, Laufzeitanalyse Baumoperationen
VL 6 „Teile und Herrsche I“: Einführung der algorithmischen Methode, Merge Sort
VL 7 „Korrektheitsbeweise“: Rechnermodel, Beispielbeweise
VL 8 „Dateien in C“: Dateien, Dateisysteme, Verzeichnisse, Dateiverwaltung mit C
VL 9 „Prioritätenschlangen/Halden/Heaps “: Heap Sort, Binärer Heap, Heap Operationen
VL 10 „Fortgeschrittene Sortierverfahren“: Quick Sort, Radix Sort
VL 11 „AVL Bäume“: Definition, Baumoperationen, Traversierung
VL 12 „Teile und Herrsche II“: Generalisierung des algorithmischen Prinzips, Mastertheorem
VL 13 „Q & A“: Offene Vorlesung/Wiederholung